2.a Kotlin Basic Syntax

A Practical Introduction to the Kotlin Language

Contents

1 **What is Kotlin?**
Introduction to the language and its key features

2 **Basic Syntax**
Hello World, Variables, Functions, String Templates

3 **Control Flow**
Conditional Expressions, Loops (for and while)

4 **Type System**
Null Safety, Type Checks, Smart Casts, Explicit Casts

5 **Collections & Lambdas**
Immutable/Mutable Collections, Lambda Expressions

6 **Classes & Inheritance**
Constructors, Data Classes, Inheritance, Open/Override

7 **Practice Exercises**
Solved examples, challenges, and tips for success

What is Kotlin?

Kotlin is a **modern, concise, and safe** programming language designed to be fully interoperable with Java.

K

Created by JetBrains

Modern language first released in 2011, now backed by the Kotlin Foundation.

A

Android First

Official language for Android development since Google's endorsement in 2017.

J

JVM Interoperability

Runs on the JVM and is 100% interoperable with existing Java code and libraries.

S

Concise & Safe

Reduces boilerplate code and eliminates entire categories of errors at compile time.

M

Multi-Platform

Compiles to JVM bytecode, JavaScript, and native binaries for cross-platform reach.

F

OOP + Functional

Supports both object-oriented and functional paradigms with first-class functions.

Hello World & Entry Point

Example

```kotlin
fun main() {
    println("Hello, World!")
    println(42)
}
```

- The **main()** function is the entry point of every Kotlin application.
- **println()** prints to standard output with a newline.
- No semicolons needed at end of statements!
- No class wrapper required (unlike Java).

```
Output:
Hello, World!
42
```

Variables: val vs var

val — Immutable (read-only)

```
val x: Int = 5
val name = "Kotlin"  // type inferred

// x = 10  ERROR! Cannot reassign
```

var — Mutable (reassignable)

```
var x: Int = 5
x += 1  // now x = 6

var msg = "Hello"
msg = "World"  // OK!
```

Key Points:

- Kotlin supports type inference — the compiler determines the type automatically
- Prefer val over var — immutability prevents bugs
- Type annotation is optional when an initializer is provided

Functions

Standard Function

```
fun sum(a: Int, b: Int): Int {
    return a + b
}
```

Expression Body (shorthand)

```
fun sum(a: Int, b: Int) = a + b

// Return type is inferred!
```

Unit Return Type (void equivalent)

```
fun greet(name: String): Unit {
    println("Hello, $name!")
}
// Unit can be omitted
```

Function Syntax:

fun name(params): ReturnType { }

- fun keyword declares a function
- Parameters: name: Type
- Return type after colon
- Unit = no meaningful return

String Templates

Example

```
var a = 1
val s1 = "a is $a"

a = 2
val s2 = "${s1.replace("is","was")}, but now is $a"

println(s2)
```

```
Output: a was 1, but now is 2
```

How it Works:

$variable inserts the variable value directly into a string.

${expression} evaluates any expression inside the braces.

No string concatenation with + needed!

Conditional Expressions

if as an Expression

```
fun maxOf(a: Int, b: Int): Int {
    if (a > b) return a
    else return b
}

// Shorthand:
fun maxOf(a: Int, b: Int) =
    if (a > b) a else b
```

when Expression (like switch)

```
fun describe(obj: Any): String =
  when (obj) {
    1           -> "One"
    "Hello"     -> "Greeting"
    is Long     -> "Long type"
    !is String  -> "Not a string"
    else        -> "Unknown"
  }
```

Key Differences from Java:

- if is an expression in Kotlin — it returns a value, so no ternary operator (? :) needed
- when replaces Java's switch — much more powerful: supports type checks, ranges, conditions
- Both if and when can be used as expressions (assigned to variables)

Loops: for and while

for Loop

```
val items = listOf("apple",
  "banana", "kiwi")

// Iterate over items
for (item in items) {
    println(item)
} // apple  banana  kiwi (one per line (OPL))

// Iterate with index
for (i in items.indices) {
    println("$i: ${items[i]}")
} // 0: apple  1: banana  2: kiwi (OPL)
```

while Loop

```
var x = 10
while (x > 0) {
    x--
}

// Ranges
for (i in 1..5) {
    print("$i ")  // 1 2 3 4 5
}

for (i in 5 downTo 1 step 2) {
    print("$i ")  // 5 3 1
}
```

- for iterates over anything with an iterator (lists, ranges, arrays, maps)
- Ranges: 1..5 (inclusive), 1 until 5 (exclusive), downTo, step
- while and do..while work the same as in Java / C

Null Safety

Nullable Types with ?

```
// Non-nullable — cannot be null
var name: String = "Kotlin"
// name = null  COMPILE ERROR

// Nullable — add ? to type
var name2: String? = "Kotlin"
name2 = null  // OK
```

Safe Calls & Elvis Operator

```
// Safe call operator ?.
val len = name2?.length
// Returns null if name2 is null

// Elvis operator ?:
val l = name2?.length ?: 0
// Returns 0 if name2 is null
```

Why This Matters:

- NullPointerException is the #1 cause of crashes in Java — Kotlin eliminates it at compile time
- All types are non-nullable by default; add ? to allow null
- ?. safely accesses properties; ?: provides a default value if null
- !! forces non-null (throws exception if null — use sparingly!)

Type Checks & Smart Casts

The is Operator (type check)

```
fun getLength(obj: Any): Int? {
    if (obj is String) {
        // obj is auto-cast to String!
        return obj.length
    }
    return null
}

println(getLength("Kotlin")) // 6
println(getLength(42))       // null
```

Smart Cast with when

```
fun describe(obj: Any): String =
  when (obj) {
    is Int    -> "Int: ${obj + 1}"
    is String -> "String: len=${obj.length}"
    is List<*>-> "List: size=${obj.size}"
    else      -> "Unknown"
  }

println(describe("Hi"))   // String: len=2
println(describe(listOf(1,2))) // List: size=2
```

How Smart Casts Work:

- After an is check, the compiler automatically casts the variable — no explicit cast needed
- Works inside if blocks, when branches, and with && / || operators
- Replaces Java's verbose pattern: if (x instanceof String) { String s = (String) x; ... }
- !is checks the opposite: if (obj !is String) return

Explicit Casts: as & as?

Unsafe Cast: as

```kotlin
val obj: Any = "Hello Kotlin"

// Works — obj is a String
val str: String = obj as String

// Throws ClassCastException!
val num: Int = obj as Int
```

Safe Cast: as?

```kotlin
val obj: Any = "Hello Kotlin"

// Returns null instead of crashing
val num: Int? = obj as? Int
println(num) // null

val str: String? = obj as? String
println(str) // Hello Kotlin
```

Java vs Kotlin — No More Verbose Casts

Java (verbose)

```java
if (obj instanceof String) {
    String s = (String) obj;
    System.out.println(s.length());
}
```

Kotlin (smart cast)

```kotlin
if (obj is String) {
    println(obj.length) // auto-cast!
}
```

Collections & Lambdas

Collections

```
// Immutable list
val fruits = listOf("apple","banana")

// Mutable list
val nums = mutableListOf(1, 2, 3)
nums.add(4)

// Map
val map = mapOf("a" to 1, "b" to 2)
```

Checking Membership

```
val items = listOf("apple", "banana")

when {
    "orange" in items -> println("juicy")
    "apple" in items  -> println("OK")
}
```

Lambda Expressions

```
val fruits = listOf("banana",
    "avocado", "apple", "kiwi")

fruits
  .filter { it.startsWith("a") }
  .sortedBy { it }
  .map { it.uppercase() }
  .forEach { println(it) }
```

```
Output:
APPLE
```

it is the implicit name for a single lambda parameter.
filter, map, sortedBy are collection functions.

Classes & Inheritance

Defining Classes

```
// Class with primary constructor
class Person(
    val name: String,
    var age: Int
)

val p = Person("Alice", 30)
println(p.name)  // Alice
p.age = 31       // OK (var)
```

Data Classes

```
// Auto-generates equals, hashCode,
// toString, copy
data class User(
    val name: String, val id: Int
)
```

Inheritance

```
// open allows inheritance
open class Shape {
    open fun area(): Double = 0.0
}

class Circle(val r: Double)
    : Shape() {
    override fun area() =
        3.14 * r * r
}
```

Key Differences from Java:

- Classes are final by default — use open to allow inheritance
- Primary constructor in class header
- data class auto-generates useful methods

Practice Exercises

Solved examples followed by exercises for you to try

Solved Exercise 1: FizzBuzz

Problem: Print numbers 1-20. For multiples of 3 print "Fizz", multiples of 5 print "Buzz", both print "FizzBuzz".

Solution

```
fun main() {
    for (i in 1..20) {
        val result = when {
            i % 15 == 0 -> "FizzBuzz"
            i % 3 == 0  -> "Fizz"
            i % 5 == 0  -> "Buzz"
            else        -> "$i"
        }
        println(result)
    }
}
```

Concepts Used:

- for loop with range (1..20)
- when expression as a value
- Modulo operator (%)
- String template ($i)
- val for immutable result

Solved Exercise 2: Safe String Length

Problem: Write a function that safely returns the length of a nullable string, or -1 if the string is null.

Solution

```kotlin
fun safeLength(s: String?): Int {
    return s?.length ?: -1
}

fun main() {
    println(safeLength("Kotlin")) // 6
    println(safeLength(null))     // -1
    println(safeLength(""))       // 0
}
```

Concepts Used:

- Nullable parameter (String?)
- Safe call operator (?.)
- Elvis operator (?:) for default
- Expression body function

```
Output:
6
-1
0
```

Solved Exercise 3: Filter & Transform

Problem: Given a list of names, filter those with 5+ characters, convert to uppercase, and sort alphabetically.

Solution

```
fun main() {
    val names = listOf("Alice", "Bob",
        "Charlie", "Dave", "Eleanor")

    val result = names
        .filter { it.length >= 5 }
        .map { it.uppercase() }
        .sorted()

    result.forEach { println(it) }
}
```

Concepts Used:

- listOf() for immutable list
- filter {} with lambda
- map {} to transform items
- sorted() for ordering
- Method chaining

```
Output:
ALICE
CHARLIE
ELEANOR
```

Your Turn! Exercises to Solve

1 Temperature Converter

Write a function celsiusToFahrenheit(c: Double) that converts Celsius to Fahrenheit.

Formula: F = C * 9/5 + 32

Hint: Use expression body syntax.

2 Grade Calculator

Write a function that takes a score (Int) and returns a grade using when:
90-100: "A", 80-89: "B", 70-79: "C", 60-69: "D", Below 60: "F"

Hint: Use ranges in when.

3 Null-Safe User Greeting

Write a function greetUser(name: String?) that returns "Hello, [name]!" or "Hello, Guest!" if name is null.

Hint: Use the Elvis operator (?:)

4 Word Counter

Given a list of sentences (List<String>), use lambdas to find the total number of words across all sentences.

Hint: Use flatMap, split, and count/size.

Challenge Exercises

5 Data Class & Inheritance

Create a data class Student(name, grade, gpa). Create a list of 5 students. Use lambdas to:

- Find students with GPA > 3.5
- Group students by grade
- Find the student with highest GPA

6 Simple Calculator

Write a calculator that reads two numbers and an operator (+, -, *, /) from the user and prints the result.

Use when for the operator, handle division by zero with null safety.

Tips for Success

- Try writing your solution before looking at reference material
- Use the Kotlin Playground at play.kotlinlang.org to test your code
- Start simple, then refactor using Kotlin idioms (when, lambdas, etc.)